

Performance Testing

Date	1 July, 2026
Team ID	
Project Name	Machine Learning-Based Flood Prediction and Early Warning System
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter, Postman
Type of Testing	Load Testing, Stress Testing
Target Module	Flask Prediction API (/predict), Flood Risk Prediction Form
Test Environment	Windows 11, Python 3.11, Flask, XGBoost Model
Test Date	7 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of virtual Users	Duration (sec)	Expected Outcome
1	Single user submits weather data for flood prediction	1	30	Floodprediction generated successfully within 2 seconds
2	Continuous flood prediction requests	100	120	Stable response time with no crashes
3	Peak load testing during heavy usage	200	180	Application remains responsive and accurately predicts flood risk

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.21 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.18 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	>40 req/sec	55 req/sec	Pass	Good request handling capacity
4	Error Rate	<1%	0%	Pass	No request failures
5	CPU Utilization	<80%	59%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	54%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The Flask application handled multiple prediction requests efficiently without performance degradation.
- Average prediction response time remained below two seconds.
- No failed prediction requests were observed during testing.
- Flask application remained stable under moderate and heavy workloads.
- Machine learning model delivered accurate predictions without performance degradation.

Bottlenecks Identified

- A slight increase in response time was observed when more than **200 concurrent users** submitted flood prediction requests simultaneously.
- Initial model loading caused a slight delay during the first prediction request.

Optimization Steps Taken

- Optimized Flask routing and request handling.
- Reduced unnecessary preprocessing operations.
- Loaded the trained ML model once during application startup.
- Enabled efficient NumPy operations for faster predictions.
- Improved input validation to reduce processing overhead.